

NAMA : FAFIQ LUTFI AZANA
KELAS : TI-1G
NIM : 254107020058

praktikum jobsheet 5

Percobaan 1

Class Faktorial

code : `package BruteForceDivideConquer.minggu5;`

```
public class Faktorial {  
    int faktorialBF(int n) {  
        int fakto =1;  
        for (int i = 1; i <= n; i++) {  
            fakto = fakto * i;  
        }  
        return fakto;  
    }  
  
    int faktorialDC(int n) {  
        if (n == 1) {  
            return 1;  
        } else {  
            int fakto = n * faktorialDC(n - 1);  
            return fakto;  
        }  
    }  
}
```

Class MainFaktorial

code : `package BruteForceDivideConquer.minggu5;`
`import java.util.Scanner;`

```
public class MainFaktorial {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Masukan nilai: ");  
        int nilai = sc.nextInt();  
  
        Faktorial fk = new Faktorial();  
        System.out.println("Nilai Faktorial " + nilai + " Menggunakan BF: " +fk.faktorialBF(nilai));  
        System.out.println("Nilai Faktorial " + nilai + " Menggunakan DC: " + fk.faktorialDC(nilai));  
        sc.close();  
    }  
}
```

Verifikasi Percobaan 1:

```
fa Open folder in new window (ctrl + click) :~/Documents/github repo/PraktikumAlgoritmaDanStrukturData$ /usr/
p /home/fafiq/.config/Code/User/workspaceStorage/2625f1a98c94fc6b8f91a2ee865255a7/redhat.java/jdt_ws
Masukan nilai: 5
Nilai Faktorial 5 Menggunakan BF: 120
Nilai Faktorial 5 Menggunakan DC: 120
fafiq@fafiq-IdeaPad-Slim-5-14AKP10:~/Documents/github repo/PraktikumAlgoritmaDanStrukturData$
```

Pertanyaan

1. Pada bagian if ($n == 1$) program melakukan pengecekan apakah n bernilai 1. jika true, maka program akan me return 1 karena faktorial 1 adalah 1.
Jika tidak, maka program akan menjalankan else yang akan melakukan penghitungan $n * \text{faktorialDC}(n - 1)$ yang berarti melakukan pemanggilan fungsi itu sendiri sehingga program akan menghasilkan nilai faktorial dari n .
2. Penggunaan perulangan selain for dapat dilakukan dengan cara menggunakan while seperti berikut :

```
int faktorialBF(int n) {
    int fakto = 1;
    int i = 1;
    while (i <= n) {
        fakto *= i;
        i++;
    }
    return fakto;
}
```

perulangan akan dilakukan selama $i \leq n$. setiap perulangan dijalankan, nilai i akan diincrement.
3. Perbedaan antara $\text{fakto} *= i$ dan $\text{fakto} = n * \text{faktorialDC}(n - 1)$
 $\text{fakto} *= i$ memerlukan perulangan sebanyak n . Sedangkan $\text{fakto} = n * \text{faktorialDC}(n - 1)$ melakukan rekursi dengan cara memanggil dirinya sendiri. Secara matematis dapat dituliskan seperti ini : $f(n) = n * f(n - 1)$.
4. Cara kerja faktorialDC adalah dengan melakukan rekursif, sedangkan faktorialBF melakukan perulangan langsung.

Percobaan 2

Class Pangkat

Code: `package BruteForceDivideConquer.minggu5;`

```
public class Pangkat {
    int nilai, pangkat;
    Pangkat() {
    }
    Pangkat(int n, int p) {
        nilai = n;
        pangkat = p;
    }
    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
    }
}
```

```

        return hasil;
    }

    int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
            } else {
                return (pangkatDC(a, n/2) * pangkatDC(a, n/2));
            }
        }
    }
}

```

Class MainPangkat

Code:

```

package BruteForceDivideConquer.minggu5;
import java.util.Scanner;

public class MainPangkat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukan jumlah elemen: ");
        int elemen = sc.nextInt();

        Pangkat[] png = new Pangkat[elemen];
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukan nilai basis elemen ke-" + (i + 1) + ": ");
            int basis = sc.nextInt();
            System.out.print("Masukan nilai pangkat elemen ke-" + (i + 1) + ": ");
            int pangkat = sc.nextInt();
            png[i] = new Pangkat(basis, pangkat);
        }

        System.out.println("HASIL PANGKAT BRUTEFORCE: ");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF(p.nilai,
p.pangkat));
        }
        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER: ");
        for (Pangkat p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatDC(p.nilai,
p.pangkat));
        }
        sc.close();
    }
}

```

Verifikasi Hasil Percobaan 2:

```
fafiq@fafiq-IdeaPad-Slim-5-14AKP10:~/Documents/github repo/Praktiku
p /home/fafiq/.config/Code/User/workspaceStorage/2625f1a98c94fc6b8f
Masukan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
fafiq@fafiq-IdeaPad-Slim-5-14AKP10:~/Documents/github repo/Praktiku
```

Pertanyaan

1. Method PangkatBF menggunakan perulangan untuk menghitung pangkat, sedangkan PangkatDC menggunakan rekursi untuk menghitung pangkat.
2. Tahap combine ada pada baris kode `return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);` dan `return (pangkatDC(a, n/2) * pangkatDC(a, n/2));`
3. Penggunaan parameter pada method PangkatBF masih relevan karena ketika method menggunakan parameter, method tersebut menjadi fleksibel sehingga method bisa digunakan untuk nilai lain dan bukan hanya pada atribut objek saja. Method tersebut bisa dibuat menjadi tanpa parameter seperti berikut

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
        hasil *= nilai;
    }
    return hasil;
}
```

4. Cara kerja method pangkatBF adalah dengan menggunakan perulangan langsung (for) yang bekerja dengan cara mengalikan satu per satu. Sedangkan pada method pangkatDC bekerja dengan melakukan rekursi dan membagi pekerjaan menjadi 2. Jika ditulis dengan notasi matematika menjadi $a^n = (a^{n/2}) * (a^{n/2})$. $2^4 = (2^2) * (2^2) = 4 * 4 = 16$.

Percobaan 3

Class Sum

code: `package BruteForceDivideConquer.minggu5;`

```
public class Sum {  
    double keuntungan[];  
  
    Sum(){}  
    Sum(int el){  
        keuntungan = new double[el];  
    }  
    double totalBF() {  
        double total = 0;  
        for (int i = 0; i < keuntungan.length; i++) {  
            total = total + keuntungan[i];  
        }  
        return total;  
    }  
}
```

```
double totalDC(double arr[], int l, int r) {  
    if (l == r) {  
        return arr[l];  
    }  
}
```

```
int mid = (l + r) / 2;  
double lsum = totalDC(arr, l, mid);  
double rsum = totalDC(arr, mid + 1, r);  
return lsum + rsum;  
}  
}
```

Class MainSum

Code:

```
package BruteForceDivideConquer.minggu5;  
import java.util.Scanner;
```

```
public class MainSum {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Masukan jumlah elemen: ");  
        int elemen = sc.nextInt();
```

```
        Sum sm = new Sum(elemen);  
        for (int i = 0; i < elemen; i++) {  
            System.out.print("Masukan keuntungan ke-" + (i + 1) + ": ");  
            sm.keuntungan[i] = sc.nextDouble();  
        }  
    }  
}
```

```
        System.out.println("Total keuntungan menggunakan BruteForce: " + sm.totalBF());  
        System.out.println("Total keuntungan menggunakan Divide and Conquer: " +  
            sm.totalDC(sm.keuntungan, 0, elemen-1));
```

```
sc.close();  
}  
}
```

Verifikasi hasil percobaan 3

```
fafiq@fafiq-IdeaPad-Slim-5-14AKP10:~/Documents/github_repo/PraktikumAlgoritmaDanStrukturData$ cd /home  
/java-21-openjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/fafiq/.config/Code/Us  
anStrukturData_268f20b5/bin BruteForceDivideConquer.minggu5.MainSum  
Masukan jumlah elemen: 5  
Masukan keuntungan ke-1: 10  
Masukan keuntungan ke-2: 20  
Masukan keuntungan ke-3: 30  
Masukan keuntungan ke-4: 40  
Masukan keuntungan ke-5: 50  
Total keuntungan menggunakan BruteForce: 150.0  
Total keuntungan menggunakan Divide and Conquer: 150.0  
fafiq@fafiq-IdeaPad-Slim-5-14AKP10:~/Documents/github_repo/PraktikumAlgoritmaDanStrukturData$
```

Pertanyaan

1. Karena untuk membagi tugas menjadi kiri dan kanan, maka dibutuhkan range untuk tugas kiri dan kanan. Jadi tugas kiri dimulai dari adalah index 0 sampai index mid, dan tugas kanan adalah dimulai dari index mid + 1 sampai index kanan.
2. Statement tersebut digunakan untuk fase conquer atau menyelesaikan. Baris tersebut melakukan penghitungan tugas kiri dan kanan.
3. Penjumlahan tersebut diperlukan untuk proses combine dimana hasil dari kedua tugas digabungkan menjadi satu melengkapi proses divide and conquer.
4. Base case nya adalah ketika $l = r$ artinya index l dan r sama yang artinya hanya ada satu angka di dalam array.
5. Cara kerja totalDC adalah dengan membagi tugas menjadi kiri dan kanan. Kemudian menghitung kiri dan kanan lalu menggabungkan hasilnya.